实验七　用 Java Socket 实现进程间通信

学号：　20002221　姓名：　赵逸翔　班级：　软件 202　成绩：

组号：　3　同组成员（2 人一组）：　王卿

实验时间　**2023.05.17**　实验地点　**信息楼 320**　指导教师　**黄建华**

一．实验目的与要求

通过本次实验，让学生能够了解如何通过Java Socket网络编程实现客户机与服务器的通信。即实现两个进程之间相互通讯。

二．实验方法与要求

网络编程就是两台计算机的进程之间相互通讯交换数据。Java SDK提供一些相对简单的Api来完成这些工作，Socket就是其中之一。

对于Java而言。这些Api存在于Java.net这个包里面。因此只要导入这个包就可以准备网络编程了。网络编程的基本模型就是客户机到服务器模型。简单的说就是两个进程之间相互通讯，其中一个必须提供一个固定的位置，而另一个则只需要知道这个固定的位置，并去建立两者之间的联系。然后完成数据的通讯就可以了。这里提供固定位置的通常称为服务器，而建立联系的通常叫做客户端。基于这个简单的模型，就可以进入网络编程。

Java对这个模型的支持有很多种Api，而这里我只介绍有关Socket的编程接口。对于Java而言已经简化了Socket的编程接口。首先我们来讨论有关提供固定位置的服务方是如何建立的。Java提供了ServerSocket来对其进行支持。事实上当你创建该类的一个实例对象并提供一个端口资源你就建立了一个固定位置可以让其他计算机来访问你。

> 以下是引用片段：
>
> ```
> ServerSocket server=new ServerSocket(6789);
> ```

这里稍微要注意的是端口的分配必须是唯一的。因为端口是为了唯一标识每台计算机唯一服务的。另外端口号是从0~65535之间的，前1024个端口已经被TCP/IP 作为保留端口，因此你所分配的端口只能是1024之后的。

好了，我们有了固定位置。现在所需要的就是一根连接线了。该连接线由客户方首先提

出要求。因此Java同样提供了一个Socket对象来对其进行支持。只要客户方创建一个Socket的实例对象进行支持就可以了。

```
以下是引用片段：
Socket client=new Socket(InetAddress.getLocalHost()，5678);
```

客户机必须知道有关服务器的IP地址。对于这一点Java也提供了一个相关的类InetAddress该对象的实例必须通过它的静态方法来提供。它的静态方法主要提供得到本机IP和通过名字或IP直接得到InetAddress的方法。

使用上面的方法基本可以建立一条连线让两台计算机相互交流了。可是数据是如何传输的呢?事实上I/O操作总是和网络编程息息相关的。所以你也必须导入Java.io这个包。Java的IO操作也不复杂。它提供了针对于字节流和Unicode的读者和写者，然后也提供了一个缓冲用于数据的读写。

```
以下是引用片段：
BufferedReader in=new BufferedReader(new InputStreamReader(server.getInputStream()));
PrintWriter out=new PrintWriter(server.getOutputStream());
```

上面两句就是建立缓冲并把原始的字节流转变为Unicode的操作。而原始的字节流来源于Socket的两个方法，getInputStream()和getOutputStream()方，分别用来得到输入和输出。那么现在有了基本的模型和基本的操作工具，就可以做一个简单的Socket例程了。

三.实验步骤：

1. 程序编写

我们可以在eclipse编程工具中进行编程，也可以直接使用记事本进行编程。

使用记事本进行编程的方法如下：

(1) 开始——运行——输入notepad，打开记事本

(2) 写入服务器端的代码，完成后将该文档保存到C:\Documents and Settings\Administrator路径下，文件名为Server.java ，保存类型为：所有类型

(3) 用同样的方法（以上1、2所述方法）对客户端代码进行操作，保存文件名为Client.java ，将Client.java中的host中的IP地址改为服务器的IP地址，重新保存后编译运行保存类型为：所有类型

2 编译运行

以上两段代码写好以后我们就可以进行编译了：

(1) 作为服务器端的PC1，开始——运行——cmd——输入javac Server.java　对服务器端进行编译，如果编译没有出错的话就进行运行：输入java Server　　如果没报错服务器端便运行成功了

（若出现编码问题，编译时使用：　javac -encoding UTF-8 XXX.java）

(2) 作为客户端的PC2重复第1条操作步骤进行客户端的编译和运行，所需命令如下：

编译：javac Client.java

运行：java　Client

如果没有出现错误便运行成功了，客户机端可以输入数据，服务器端可以收到来自客户端的数据

(3) 注意：在输入DOS命令时要注意文件名的大小写，同时要注意空格的输入

使用eclipse进行编程方法如下：

打开Eclipse

File——New——Project(命名，Finish)

新建的Project——New——Class(命名，Server)：写入服务端代码

新建的Project——New——Class(命名，Client)：写入客户端代码，将host中的IP地址改为服务器的IP地址，运行程序

Eclipse中用两个控制台测试网络通信程序

https://blog.csdn.net/f309587969/article/details/7097976

3 测试：

（1）在客户端的DOS窗口中输入字符串，回车，服务器端便能收到来自客户端的字符。如果客户端想要退出，则可以直接输入end

（2）在Eclipse中客户端程序运行的调试控制台输入字符串，服务器端便能收到来自客户端的字符

4．具体实验代码如下：

服务器代码：

```
/**
 * Created by xl on 2017/4/19.
```

```
 */
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;
public class Server {
    public static void main(String[] args) throws IOException {
        //为了简单起见，所有的异常信息都往外抛
        int port = 8888;
        //定义一个 ServerSocket 监听在端口 8888 上
        ServerSocket server = new ServerSocket(port);
        while (true) {
            //server 尝试接收其他 Socket 的连接请求，server 的 accept 方法是
阻塞式的
            Socket socket = server.accept();
            //每接收到一个 Socket 就建立一个新的线程来处理它
            new Thread(new Task(socket)).start();
        }
    }


    static class Task implements Runnable {

        private Socket socket;

        public Task(Socket socket) {
            this.socket = socket;
        }

        public void run() {
            try {
                handleSocket();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }

        /**
         * 跟客户端Socket 进行通信
         *
         * @throws Exception
         */
```

```
        private void handleSocket() throws Exception {
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream());
            System.out.println(socket.getInetAddress());
            while (true) {
                String str = in.readLine();
                System.out.println(str);
                out.println("has receive...");
                out.flush();
                if (str.equals("end"))
                    break;
            }
            socket.close();
        }

    }
}
```

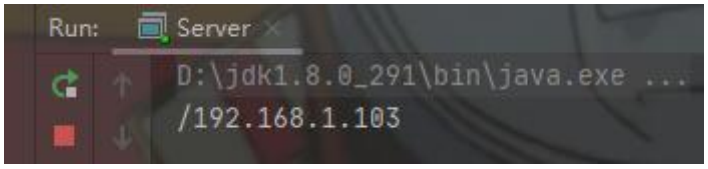
客户机代码:

```
/**
 * Created by xl on 2017/4/19.
 */
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;

public class Client {
    public static void main(String[] args) throws IOException {
        //为了简单起见，所有的异常都直接往外抛
        String host = "192.168.0.244";  //要连接的服务端 IP 地址
        int port = 8888;   //要连接的服务端对应的监听端口
        //与服务端建立连接
        Socket client = new Socket(host, port);
        //建立连接后就可以往服务端写数据了
        BufferedReader in=new BufferedReader(new
InputStreamReader(client.getInputStream()));
        PrintWriter out=new PrintWriter(client.getOutputStream());
        BufferedReader wt=new BufferedReader(new
InputStreamReader(System.in));
        while(true){
            System.out.println("请输入:");
```

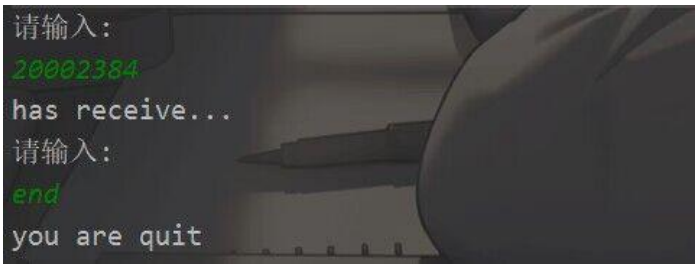
```
            String str=wt.readLine();
            out.println(str);
            out.flush();
            if(str.equals("end"))
            {
                break;
            }
            System.out.println(in.readLine());
        }

        client.close();
        System.out.println("you are quit");
    }

}
```

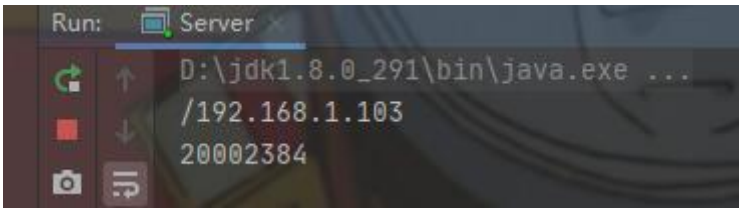
5. 实验效果截图

（1）客户机首先连接服务器，给出服务器端显示客户端 IP 地址的截图



（2）客户端输入自己的学号，给出客户端输入消息和服务器端收到消息的效果截图





四．思考题

1、如何实现socket对等通信，即既可以作为服务器端，也可以作为客户端?

答：服务器端SocketServer使用accept()，同时两端使用OutputStream和InputStream获取输入输出流，以下是一种实现方法。

服务器端：

```
package server;


import java.io.IOException;

import java.io.InputStream;

import java.io.OutputStream;

import java.net.ServerSocket;

import java.net.Socket;

import java.util.Scanner;


public class SocketServer {

    public static void main(String[] args) {


        try {

            final ServerSocket serverSocket = new ServerSocket(8802);

            System.out.println("服务器端 socket 套接字创建成功...");


            //循环使得客户端能够一直交互

            while (true){

                //客户端接入

                final Socket con = serverSocket.accept();
```

```
System.out.println("客户端接入...");
int i = 10;

while (i > 0){
    new Thread(new Runnable() {
        public void run() {
            //获取输入流及输出
            InputStream inputStream = null;
            try {
                inputStream = con.getInputStream();
                int len = 0;
                byte[] data = new byte[256];
                len = inputStream.read(data);
                System.out.println("读到消息：" + new String(data, 0,
len));

                OutputStream outputStream = con.getOutputStream();
                Scanner sc = new Scanner(System.in);
                //获取字符串
                String message = sc.next();

                //发送通道，用于发送数据
```

```
                    outputStream.write(message.getBytes());
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        }).start();
        i--;
    }


    }
} catch (IOException e) {
    e.printStackTrace();
}
}
}
```

客户端：

```
package client;


import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;
```

```java
import java.util.Scanner;


public class SocketClient {
    public static void main(String[] args) {
        try {
            Socket clientSocket = new Socket("192.168.56.1", 8802);


            while (true){
                OutputStream outputStream = clientSocket.getOutputStream();
                Scanner sc = new Scanner(System.in);
                //获取字符串
                String message = sc.next();


                //发送通道，用于发送数据
                outputStream.write(message.getBytes());


                //接收通道
                InputStream inputStream = clientSocket.getInputStream();


                int len;
                byte[] data = new byte[256];
                //接收数据
```

```
                len = inputStream.read(data);
                System.out.println("获取服务端返回消息：" + new String(data, 0, len));
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

2、如何实现多个客户同时访问一个服务器?

答：可以使用多线程实现，使多个客户机并发访问服务器。